Variables and Data types

Variables are like a labelled box for storing and referencing data of different types.

To declare variables in Python, you assign a value to an identifier with the assignment (`=`) operator.

When naming variables in Python, there are some important rules you should keep in mind:

- Variable names can only start with a letter or an underscore (`_`), not a number.
- Variable names can only contain alphanumeric characters (`a-z`, `A-Z`, `0-9`) and underscores (`_`).
- Variable names are case-sensitive — `age`, `Age`, and `AGE` are all considered unique.
- Variable names cannot be one of Python's reserved keywords such as `if`, `class`, or `def`.

If you break any of those rules, your Python program will throw a `SyntaxError`

A common naming convention is that, variable names should be in lowercase, with separate words separated by an underscore (_).

you can use the `print` function to print data to the terminal.

A data type describes the kind of value a variable holds. For example, a number, a piece of text, or a list of items. Programming languages use data types so they know how to store and work with different kinds of information.

The dynamic-typing nature of Python makes coding really fast and more flexible, but it can lead to unexpected bugs because type errors are detected only when a program runs, not when the program compiles.

When a program runs, Python executes your code line by line. If it reaches a line where a certain object is expected to behave in a way it's not able to, Python will stop and show an error.

Common data types you'll use in Python:

- **Integer:** A whole number without decimals, for example, `10` or `-5`.
- **Float:** A number with a decimal point, like `4.41` or `-0.4`.
- **String:** A sequence of characters enclosed in single or double quotation marks like `'Hello world!'`.
- **Boolean:** A true or false type, written as `True` or `False`.
- **Set:** An unordered collection of unique elements, like `{0.5, 4, 'apple'}`.
- **Dictionary:** A collection of key-value pairs enclosed in curly braces, like `{'name': 'John Doe', 'age': 28}`.
- **Tuple:** An immutable ordered collection, enclosed in parentheses, like `('apple', 4.5, 7)`.
- **List:** An ordered collection of elements that supports different data types.
- **None:** A special value that represents the absence of a value.

To get the data type of a variable, you can use the `type()` function:

```
my_name = "John Doe"
my_age = 18
is_adult = True
my_height = 170.5

print(type(my_name)) # <class 'str'>
print(type(my_age)) # <class 'int'>
print(type(is_adult)) # <class 'bool'>
print(type(my_height)) # <class 'float'>
```

The built-in `isinstance()` function lets you check if a variable matches a specific data type. It takes in an object and the type you want to check it against, then returns a boolean. Here are some examples:

```
isinstance('Hello world', str) # True
isinstance(True, bool) # True
isinstance(42, int) # True
isinstance('John Doe', int) # False
```

Strings

A string is a sequence of characters surrounded by either single or double quotation marks. In some programming languages, characters surrounded by single quotes are treated differently than characters surrounded by double quotes, but in Python, they're treated equally.

If you need a multi-line string, you can use triple double quotes or single quotes.

```
str_1 = """Multiline
string"""
str_2 = '''Another
multiline
string'''
```

If your string contains either single or double quotation marks, then you have two options:

- Use the opposite kind of quotes. That is, if your string contains single quotes, use double quotes to wrap the string, and vice versa
- Escape the single or double quotation mark in the string with a backslash (`\`). With this method, you can use either single or double quotation marks to wrap the string itself

Python provides the `in` operator, which returns a boolean that specifies whether the character or characters exist in the string or not.

```
my_str = 'Python Programming'

print('Python' in my_str)  # True
print('hello' in my_str)    # False
print('gram' in my_str)    # True
print('ht' in my_str)  # False
print(' ' in my_str)  # True
```

To get the length of a string, you can use the built-in `len()` function.

```
print(len(my_str)) # 18
```

Each character in a string has a position called an index. The index is zero-based, meaning that the index of the first character of a string is `0`, the index of the second character is `1`, and so on. To access a character by its index, you use square brackets (`[]`) with the index of the character you want to access inside. Negative indexing is also allowed, so you can get the last character of any string with `-1`, the second-to-last character with `-2`, and so on.

Indexing is the process of accessing an individual element of a collection (such as a string or list) using its position (index).

```
print(my_str[0])  # P
print(my_str[5])  # n
print(my_str[-1])  # g
print(my_str[4])  # o
```

Immutable data types can't be modified or altered once they're declared. You can point their variables at something new, which is called reassignment, but you can't change the original object itself by adding, removing, or replacing any of its elements.

Strings are immutable data types in Python. This means that you can reassign a different string to a variable, but direct modification of a string isn't allowed.

```
my_str[2] = 'Y' #TypeError: 'str' object does not support item assignment
```

Examples of other immutable data types in Python are integer, float, boolean, tuple, and range. You'll get to know each of these types in upcoming lessons.

String Concatenation

You can combine multiple strings together with the plus (`+`) operator. This process is called **string concatenation**.

```
my_str_1 = 'Python'
my_str_2 = "Programming"

str_plus_str = my_str_1 + ' ' + my_str_2
```

```
print(str_plus_str) # Python Programming
```

But note that this only works with strings. If you try to concatenate a string with a number, you'll get a `TypeError.`

```
name = 'Gilbert Butler'
age = 22

name_and_age = name + age
print(name_and_age) # TypeError: can only concatenate str (not "int") to str
```

You can also use the augmented assignment operator for concatenation. This is represented by a plus and equals sign (`+=`), and performs both concatenation and assignment in one step.

```
first_name = 'Gilbert'
last_name = 'Butler'

full_name = first_name
full_name += last_name
print(full_name) # Gilbert Butler
```

String Interpolation

The process of inserting variables and expressions into a string is called **string interpolation**. Python has a category of string called **f-strings** (short for formatted string literals), which allows you to handle interpolation with a compact and readable syntax.

F-strings start with `f` (either lowercase or uppercase) before the quotes, and allow you to embed variables or expressions inside replacement fields indicated by curly braces (`{}`).

```
name = 'Gilbert Butler'
age = 23
name_and_age = f'My name is {name} and I am {age} years old'
print(name_and_age) # My name is Gilbert Butler and I am 23 years old
```

Note how you don't need to convert non-string types with the `str()` function.

String slicing lets you extract a portion of a string or work with only a specific part of it.

```
string[start:stop:step]
```

By default, step is set as 1 and does not need to be defined. If the stop is not set, it automatically sets to the end of the string.

```
my_str = 'Hello world'
print(my_str[1:4]) # ell
```

```
my_str = 'Hello world'
print(my_str[0:11:2])  # Hlowrd
```

```
my_str = 'Hello world'
print(my_str[::-1]) # dlrow olleH
```

String Methods

Python provides a number of built-in methods that make working with strings.

- `upper()`: Returns a new string with all characters converted to uppercase.
- `lower()`: Returns a new string with all characters converted to lowercase.
- `strip()`: Returns a new string with the specified leading and trailing characters removed. If no argument is passed it removes leading and trailing whitespace.
- `replace(old, new)`: Returns a new string with all occurrences of `old` replaced by `new`.
- `split(separator)`: Splits a string on a specified separator into a list of strings. If no separator is specified, it splits on whitespace.
- `startswith(prefix)`: Returns a boolean indicating if a string starts with the specified prefix.
- `endswith(suffix)`: Returns a boolean indicating if a string ends with the specified suffix.

- `find(substring)`: Returns the index of the first occurrence of `substring`, or `-1` if it doesn't find one.
- `count(substring)`: Returns the number of times a substring appears in a string.
- `capitalize()`: Returns a new string with the first character capitalized and the other characters lowercased.
- `isupper()`: Returns `True` if all letters in the string are uppercase and `False` if not.
- `islower()`: Returns `True` if all letters in the string are lowercase and `False` if not.
- `title()`: Returns a new string with the first letter of each word capitalized.

Example usage:

```
my_str = 'hello world'

uppercase_my_str = my_str.upper()
print(uppercase_my_str)  # HELLO WORLD

my_str = 'Hello World'

lowercase_my_str = my_str.lower()
print(lowercase_my_str)  # hello world

my_str = '  hello world  '

trimmed_my_str = my_str.strip()
print(trimmed_my_str)  # "hello world"

my_str = 'hello world'

replaced_my_str = my_str.replace('hello', 'hi')
print(replaced_my_str)  # hi world

my_str = 'hello world'

split_words = my_str.split()
print(split_words)  # ['hello', 'world']
```

```python
my_str = 'hello world'

starts_with_hello = my_str.startswith('hello')
print(starts_with_hello)  # True

my_str = 'hello world'

ends_with_world = my_str.endswith('world')
print(ends_with_world)  # True

my_str = 'hello world'

world_index = my_str.find('world')
print(world_index)  # 6

my_str = 'hello world'

o_count = my_str.count('o')
print(o_count)  # 2

my_str = 'hello world'

capitalized_my_str = my_str.capitalize()
print(capitalized_my_str)  # Hello world

my_str = 'hello world'

is_all_upper = my_str.isupper()
print(is_all_upper)  # False

my_str = 'hello world'

is_all_lower = my_str.islower()
print(is_all_lower)  # True

my_str = 'hello world'

title_case_my_str = my_str.title()
print(title_case_my_str)  # Hello World
```